Architectural and Mathematical Formulations for Optimizing the Hybrid Mamba-xLSTM Medical Model

1. Introduction to the Hybrid State-Space and Recurrent Paradigm

The intersection of state-space models and recurrent neural network architectures presents a highly promising frontier for biomedical vision-language modeling. The 70-million parameter Hybrid Mamba-xLSTM model, subjected to the Stage 0 language modeling evaluation on the PubMed dataset, represents a specific class of lightweight, high-efficiency architectures designed for specialized clinical domains.[1] According to the empirical evaluation metrics captured on an NVIDIA A100-SXM4-40GB GPU, the current architecture comprises eight interleaved layers following the pattern of mamba, mamba, mlstm, mamba, mamba, mlstm, mamba, mamba with a uniform hidden dimension of 512.[1] While the evaluation perplexity stabilizes at 13.10 across 1000 validation samples, indicating a baseline capacity for sequence modeling and linguistic comprehension, the model's multimodal alignment strategy exhibits critical architectural vulnerabilities.[1]

The primary objective of this comprehensive analysis is to mathematically deconstruct and architecturally resolve five specific structural gaps that currently hinder the model's ability to map medical text into a shared latent space with medical imaging representations, specifically targeting alignment with the BiomedCLIP image space. These gaps encompass the mathematical limitations of unconstrained contrastive loss in low-batch settings, the semantic dilution caused by suboptimal projection and pooling mechanisms, the stochastic divergence introduced during training phase dropout, and the gradient instability of unconstrained learnable temperatures. By addressing these issues through rigorous mathematical reformulation and hardware-aware architectural adjustments, the Hybrid Mamba-xLSTM model can achieve robust, zero-shot multimodal alignment without exceeding the memory constraints of a single-GPU pre-training environment. The ensuing sections will systematically dismantle each of these five gaps, providing theoretical grounding, mathematical proofs, and concrete architectural blueprints tailored explicitly for the 70M parameter setup.

2. Hardware Profiling and Baseline Training Constraints

Before addressing the specific mathematical formulations of the contrastive loss and architectural components, it is imperative to analyze the hardware constraints under which this model operates. The design choices for multimodal alignment cannot be abstracted away from

the physical memory and compute limitations of the target hardware. The evaluation log eval_stage0_lm_1135.log provides a precise snapshot of the operational environment.[1]

2.1 Compute Environment and Memory Footprint

The system runs on PyTorch version 2.8.0+cu128 utilizing an NVIDIA A100-SXM4-40GB GPU.[1] The SXM4 form factor allows for a maximum power draw of 400W, significantly higher than the standard PCIe variant, enabling sustained high-frequency tensor operations.[1] However, the critical bottleneck is the 40GB (39.5 GB usable) VRAM limit.

The evaluation logs reveal the presence of Multi-Instance GPU (MIG) partitioning on the secondary GPU (GPU 1), but the primary training and evaluation processes are executed on GPU 0, which has MIG disabled, granting the process access to the full 40960 MiB of VRAM.[1] During the Stage 0 LM evaluation, the peak GPU memory observed was 7.67 GB.[1] It is crucial to note that this 7.67 GB footprint represents the inference state.

During multimodal contrastive pre-training, the VRAM requirement expands exponentially. Training necessitates the storage of the forward activations for backpropagation, the gradients for each of the 83,139,328 parameters, and the optimizer states (e.g., first and second moments in AdamW, effectively tripling the parameter memory footprint).[1] When processing paired image and text inputs, the memory burden is further exacerbated by the vision encoder's activations.

2.2 Sequence Throughput and Batch Size Limitations

The evaluation metric details the throughput across various sequence lengths:

- Sequence length 128: 82,541 tokens/second (24.8 ms/batch)
- Sequence length 256: 86,984 tokens/second (47.1 ms/batch)
- Sequence length 512: 87,253 tokens/second (93.9 ms/batch)
- Sequence length 1024: 88,076 tokens/second (186.0 ms/batch) [1]

While the Mamba and xLSTM layers demonstrate excellent sub-quadratic scaling—maintaining high token throughput even at a sequence length of 1024—the spatial memory requirements dictate that the global batch size must remain relatively small to avoid Out-Of-Memory (OOM) errors.[1] In standard contrastive learning frameworks like SimCLR, optimal performance is achieved with massive batch sizes (e.g., 4096 or 8192) to provide a sufficient number of negative samples.[4] Given the 40GB constraint, a batch size of 4096 is mathematically impossible for a multimodal setup involving high-resolution medical images and long radiology reports. Consequently, the architecture must abandon standard in-batch contrastive sampling in favor of decoupled, memory-efficient mechanisms.

Metric	Measured Value	Implication for Contrastive

		Learning
Model Parameters	83.1M (83,139,328)	Lightweight enough for rapid iteration, but requires high-quality representations to avoid underfitting.
Peak Inference VRAM	7.67 GB	Training VRAM will likely exceed 30 GB with modest batch sizes.
Max Sequence Length	1024 tokens	Capable of processing full radiology reports, necessitating robust pooling mechanisms.
Hardware	A100-SXM4-40GB	High compute ceiling (400W) but strict memory bounds prevent standard large-batch NT-Xent loss.

3. Resolving Low Negative Coverage via Momentum Contrast (MoCo)

The fundamental engine of vision-language alignment is contrastive learning, which operates on the premise of minimizing the distance between positive image-text pairs while maximizing the distance between negative pairs. The standard Normalized Temperature-scaled Cross Entropy (NT-Xent) loss, widely adopted in multimodal architectures, relies heavily on in-batch negatives.[6] This dependency introduces the first major architectural gap: low negative coverage due to restricted batch sizes.

3.1 The Mathematical Bottleneck of NT-Xent

In a standard NT-Xent implementation, the contrastive loss for a positive pair is computed across a batch of size N, yielding $2(N-1)$ negative samples when generating two

augmented views per instance, or simply $N-1$ cross-modal negatives in a standard image-text batch.[5] The mathematical formulation is given by the InfoNCE objective:

$$\mathcal{L}_q = -\log \frac{\exp(q \cdot k_+/\tau)}{\sum_{i=0}^{K} \exp(q \cdot k_i/\tau)}$$

Where q represents the query embedding (e.g., the text representation), k_+ represents the positive key (the corresponding image representation), k_i represents the negative keys (unmatched images in the batch), and τ is the temperature parameter.[8]

The gradient of this loss function dictates that the model learns by pushing the query away from the negative keys. The effectiveness of this optimization scales logarithmically with the number of negative samples K.[10] If K is small (e.g., a batch size of 32 or 64 imposed by the 40GB VRAM limit), the loss landscape fails to form a uniform hypersphere.[11] The model easily minimizes the loss by separating a few trivial negatives, leading to poor generalization and a failure to learn the fine-grained semantic distinctions necessary for medical imagery.[3] In the medical domain, distinguishing between two chest X-rays that are largely identical except for a subtle 5mm nodule requires an exceptionally large and diverse pool of negatives.[13]

3.2 The Dual-Queue Momentum Architecture

To decouple the dictionary size from the hardware-constrained batch size, the architecture must integrate the Momentum Contrast (MoCo v2) mechanism.[10] MoCo resolves the negative coverage deficit by maintaining a dynamic, queue-based dictionary of precomputed key embeddings.[17]

Instead of relying solely on the current mini-batch, MoCo maintains a queue of size K (e.g., $K = 65,536$), which can be orders of magnitude larger than the mini-batch size.[10] When a new batch of medical text and images is processed, their encoded representations are enqueued, and the oldest representations are dequeued, operating strictly on a First-In-First-Out (FIFO) basis.[10] This allows the contrastive loss denominator to sum over 65,536 negative samples regardless of the local GPU batch size.[10]

3.3 Overcoming Representation Inconsistency

Using a continuously updated queue introduces a secondary challenge: representation inconsistency. Keys encoded thousands of steps ago were processed by significantly different network weights than the current queries.[10] If the key encoder is updated rapidly via standard backpropagation, the embeddings in the queue lose their geometric coherence, and the contrastive targets become moving, noisy vectors that degrade the InfoNCE optimization.[12]

To ensure dictionary consistency, MoCo introduces a momentum encoder. The system is split into two distinct networks with identical architectures: a query encoder (parameterized by θ_q) and a key encoder (parameterized by θ_k).[12] During training, only the query encoder θ_q receives gradients and is updated via the AdamW optimizer. The parameters of the key encoder θ_k are completely detached from the computational graph and are instead updated via an exponential moving average of the query encoder's weights [10]:

$$\theta_k \leftarrow m\theta_k + (1-m)\theta_q$$

Where m is a momentum coefficient meticulously tuned to remain close to 1 (e.g., $m = 0.999$ or 0.9999).[10] This momentum update ensures that the key representations in the queue evolve at a glacial pace, maintaining semantic consistency across thousands of training iterations while still gradually absorbing the structural improvements of the query network.[10]

3.4 Hardware and Implementation Benefits for the 70M Setup

For the 70M parameter Hybrid Mamba-xLSTM architecture, the MoCo v2 implementation provides profound hardware advantages. Because the key encoder is updated via momentum rather than backpropagation, gradients are not computed, tracked, or stored for the key network.[12] This effectively halves the gradient memory overhead per step, allowing the model to train efficiently on the A100 40GB GPU.

Furthermore, storing a queue of 65,536 embeddings requires negligible VRAM. Assuming the final projected dimension is 128 (as explored in subsequent sections) and using 32-bit floating-point precision, the entire memory bank consumes roughly 33 Megabytes ($65536 \times 128 \times 4$ bytes), which is essentially free relative to the 39.5 GB available.[1]

In the context of aligning text with the BiomedCLIP image space, the model will maintain a momentum queue of BiomedCLIP image embeddings. The Hybrid Mamba-xLSTM acts as the query encoder f_q, transforming radiology text into query vectors q. These are matched against the positive image key k_+ and the massive dictionary of negative image keys $\{k_i\}$.[9] This completely solves the first architectural gap, granting the model the mathematical properties of a massive-batch contrastive setup while operating strictly within a small-batch hardware footprint.[12]

4. The Necessity of a Non-Linear Projection Bridge

The second major architectural flaw in the baseline configuration is the assumption that the raw terminal states of the Mamba and xLSTM layers can be mapped directly into the

BiomedCLIP latent space using a simple linear transformation or identity mapping. Empirical research, most notably established by the SimCLR framework, explicitly demonstrates that adopting a non-linear multi-layer perceptron (MLP) projection head profoundly impacts the quality of the learned representations, often yielding performance improvements in excess of 10% on downstream linear evaluation tasks.[4]

4.1 The Information Bottleneck and Dimensional Collapse

State-space models like Mamba and recurrent models like xLSTM process sequences by continuously updating and compressing historical context into a hidden state vector. The final hidden representation h (where $h \in \mathbb{R}^{512}$ for this 70M architecture) contains rich, dense information regarding syntax, grammar, localized entity relations, and sequential text progression.[1]

The primary objective of contrastive learning is to induce invariance to noise and augmentation, pushing the representations of different views (or modalities) closer together.[4] However, this objective is inherently destructive. The contrastive loss aggressively discards information deemed irrelevant to the immediate alignment task.[4] For example, in a medical setting, the exact phrasing, grammatical structure, and localization of a text prompt may be discarded by the loss function in its attempt to match the broader semantic concept of the paired image.

When the representation h is passed directly to the contrastive loss function (an identity mapping), the destructive invariance propagates directly into the Mamba and xLSTM backbone.[4] The backbone is penalized for retaining low-level semantic details, leading to a phenomenon known as dimensional collapse. In dimensional collapse, the neural network learns to utilize only a small subspace of the 512-dimensional capacity, focusing entirely on the strongest global features while ignoring weaker, fine-grained features.[14] Consequently, the downstream utility of the text representations for tasks like masked language modeling, classification, or zero-shot inference is permanently degraded.[4]

4.2 Formulating the MLP Projection Head

To insulate the rich sequential representations from the destructive invariance of the contrastive objective, a non-linear projection head must be introduced.[4] The projection head $g(\cdot)$ acts as a learnable information bottleneck that maps the representation h to the contrastive latent space z.[4]

The mathematical definition of the non-linear projection head utilized in optimal contrastive frameworks (e.g., SimCLR, MoCo v2) is a two-layer or three-layer Multi-Layer Perceptron (MLP)[6]:

$$z = g(h) = W^{(2)}\sigma\left(\text{BatchNorm}(W^{(1)}h)\right)$$

Here, $W^{(1)}$ and $W^{(2)}$ are the learnable weight matrices of the linear layers, and σ represents a non-linear activation function, typically Rectified Linear Unit (ReLU) or Gaussian Error Linear Unit (GELU).[6]

The introduction of this non-linearity is theoretically profound. A purely linear projection $z = Wh$ can only perform a rigid rotation, scaling, and skewing of the latent space, meaning any information discarded in z is essentially destroyed in h as well.[4] Conversely, the non-linear activation σ allows the projection head to bend and fold the vector space. This allows the lower layers (the Mamba and xLSTM blocks) to learn and retain complex, specialized features in h that are completely masked or collapsed in the higher, projected space z.[33] The projection head absorbs the information loss required by the contrastive objective, preserving the structural integrity of h for downstream medical reasoning tasks.[4] Once pre-training is complete, the projection head $g(\cdot)$ is entirely discarded, and the pristine representation h is utilized for clinical inference.[4]

4.3 Dimensionality Mapping for the 70M Setup

Given the model's architectural blueprint utilizing a uniform dimension of dim=512 across its 8 layers [1], the projection head must be scaled appropriately to bridge into the BiomedCLIP latent space. BiomedCLIP typically operates with projection spaces of 256 or 512 dimensions depending on the specific visual backbone utilized (e.g., ViT-B/16).[34]

For the Hybrid Mamba-xLSTM model, an optimal configuration maps the 512-dimensional output to an intermediate hidden layer. Standard projection heads either maintain or expand the hidden dimension to enhance non-linear transformation capacity.[31]

Component	Layer Type	Input Dimension	Output Dimension	Activation / Norm
Backbone	Mamba/xLSTM	Text Sequence	512	None (Representation h)
Projector L1	Linear	512	512 (or 2048)	BatchNorm1d +

				GELU
Projector L2	Linear	512 (or 2048)	512 (or 256)	None (Latent Space z)

Table 1: Structural mapping of the proposed non-linear projection head bridging the Mamba-xLSTM backbone to the BiomedCLIP target space.

By implementing this projection head, the model satisfies the second architectural gap, ensuring that the alignment process maps the text modality into the shared space without corrupting the underlying sequence representations generated by the state-space and recurrent layers.[28]

5. Semantic Localization through Attention-Weighted Pooling

In standard natural language processing pipelines, deriving a fixed-length sentence or document embedding from a sequence of token embeddings is conventionally achieved via simple global average pooling (mean pooling) or by utilizing a specialized `` token prepended to the sequence.[2] However, applying mean pooling to radiology reports and biomedical texts results in catastrophic semantic dilution.[36]

5.1 The Linguistic Topology of Radiology Reports

Radiology reports, such as those found in PubMed, the MIMIC-CXR dataset, or CT-RATE, possess a highly idiosyncratic linguistic structure.[13] A typical diagnostic finding consists of dense, repetitive boilerplate text describing normal anatomy, interrupted by highly specific, sparse clinical entities indicating pathology.[38] Furthermore, medical language relies heavily on negation and uncertainty modifiers.[13] For instance, a report may state: "The heart size is normal. The lungs are clear. There is no evidence of pleural effusion or pneumothorax. A subtle 5mm radiopaque nodule is noted in the left upper lobe.".[13]

If mean pooling is applied to this sequence, the token embeddings for the critical finding ("subtle", "5mm", "nodule") are mathematically averaged with dozens of tokens representing negative findings ("no", "evidence", "pneumothorax") and anatomical background ("heart", "size", "normal").[13] Because normal findings drastically outnumber abnormal conditions in medical datasets, the resulting pooled vector is overwhelmingly dominated by healthy anatomical descriptors.[13] The contrastive loss function becomes effectively blind to the actual pathologies, viewing two vastly different patient reports as nearly identical because 90% of their text

consists of the same healthy boilerplate.[13] This exacerbates the false negative problem in contrastive learning.[13]

5.2 Formulating Attention-Weighted Pooling

To extract high-fidelity semantic representations that emphasize critical pathologies and negations, the architecture must abandon mean pooling and transition to the attention-weighted pooling mechanism utilized by advanced medical vision-language models like BioViL-T and CXR-BERT.[38] Attention pooling introduces a learnable, dynamic weighting scheme that evaluates the relative clinical importance of each token in the sequence before aggregation.[45]

Given a sequence of token embeddings $H =$ produced by the final Mamba/xLSTM layer (where $h_i \in \mathbb{R}^{512}$), the attention pooling layer first computes an unnormalized scalar score u_i for each token via a fully connected layer:

$$u_i = V^T \tanh(W h_i + b)$$

Here, $W \in \mathbb{R}^{d_a \times 512}$ and $V \in \mathbb{R}^{d_a \times 1}$ are learnable weight matrices, and d_a is the hidden dimension of the attention module.[47] Next, these scores are normalized across the sequence length T using a softmax function to produce the attention weights α_i:

$$\alpha_i = \frac{\exp(u_i)}{\sum_{j=1}^{T} \exp(u_j)}$$

The final, fixed-length sequence-level representation h_{pool} is calculated as the attention-weighted sum of the token embeddings [44]:

$$h_{pool} = \sum_{i=1}^{T} \alpha_i h_i$$

5.3 Synergistic Benefits for the Mamba-xLSTM Backbone

During the multimodal contrastive training process, the gradients flowing backward from the InfoNCE loss will specifically optimize the matrices W and V. The model inherently learns to assign higher attention weights ($\alpha_i \to 1$) to tokens representing clinical entities, anomalous diagnoses, and crucial modifying words (such as "decreased," "subtle," "opacity," or "no"), while aggressively down-weighting repetitive boilerplate text ($\alpha_i \to 0$).[13]

The integration of attention pooling is exceptionally synergistic with the underlying state-space

and recurrent architectures. While Transformer-based models rely on global bidirectional self-attention to route information, state-space models like Mamba and xLSTM inherently compress information left-to-right. Consequently, the final tokens in a Mamba sequence often carry a heavy inductive bias toward the end of the document. By applying a global attention-pooling layer over the entire sequence of hidden states, the model provides a "shortcut" for gradients to route directly to the most critical clinical terms regardless of their position in the text sequence.[50] This ensures that the final 512-dimensional vector is an accurate, pathology-dense representation of the entire medical report, fully resolving the third architectural gap and enabling zero-shot phrase grounding.[42]

6. Mitigating Stochastic Divergence with R-Drop Regularization

The implementation of Dropout within the MLP projection head and the underlying Mamba-xLSTM sequence layers is a standard, indispensable practice to prevent neural co-adaptation and mitigate overfitting, particularly when training on highly specialized medical datasets.[52] However, the stochastic nature of Dropout introduces a significant mathematical discrepancy between the training phase and the evaluation phase.[52]

During training, random neurons are dropped out, meaning the model evaluates an ensemble of smaller sub-networks. During inference, Dropout is deactivated, and the full network is utilized, scaled by the dropout probability. This creates a non-negligible train/inference mismatch.[52] In the context of the Hybrid Mamba-xLSTM model, this discrepancy destabilizes the highly sensitive contrastive multimodal alignment space. If the embedding of a specific radiology report fluctuates randomly during training due to dropout noise, the model struggles to establish a firm, geometric mapping to the corresponding BiomedCLIP image embedding.[53]

6.1 The Mechanics of Consistency Regularization via R-Drop

To resolve this inconsistency and stabilize the projection space, the architecture must incorporate Regularized Dropout (R-Drop).[52] R-Drop operates as an elegant, implicit contrastive learning mechanism that enforces output consistency across different Dropout samples of the exact same input.[53]

During a single training step, a given text sequence x_i is passed through the network's forward pass twice. Because Dropout is active, the two forward passes traverse entirely different computational sub-networks (different dropout masks), yielding two distinct probability distributions or normalized projection embeddings, denoted as $P_1(y_i|x_i)$ and $P_2(y_i|x_i)$.[52] R-Drop asserts that despite the internal noise, the final semantic representation of the same input should remain identical.[58]

6.2 Mathematical Formulation of Bidirectional KL Divergence

R-Drop constrains the representational variance by applying a bidirectional Kullback-Leibler (KL) divergence penalty between the two distinct outputs.[58] To understand this mathematically, one must look at the foundation of information theory. The standard KL divergence, which measures the information lost when approximating a true distribution P with an approximated distribution Q, is derived from the difference between Cross-Entropy $H(P, Q)$ and Entropy $H(P)$ [60]:

$$\mathcal{D}_{KL}(P \parallel Q) = H(P, Q) - H(P) = -\sum P(x) \log Q(x) - \left(-\sum P(x) \log P(x)\right)$$

$$\mathcal{D}_{KL}(P \parallel Q) = \sum P(x) \log \frac{P(x)}{Q(x)}$$

[62]

Because standard KL divergence is asymmetric (i.e., $\mathcal{D}_{KL}(P \parallel Q) \neq \mathcal{D}_{KL}(Q \parallel P)$), it cannot be used as a balanced regularizer.[62] Therefore, R-Drop calculates the symmetric, bidirectional loss [52]:

\mathcal{L}_{KL}^i = \frac{1}{2} \left

This KL divergence penalty is scaled by a coefficient α and added to the primary contrastive loss (the InfoNCE loss over the MoCo queue) [57]:

$$\mathcal{L}_{total} = \mathcal{L}_{InfoNCE} + \alpha \mathcal{L}_{KL}^i$$

6.3 Implications for the Projection Head and VRAM

Applying R-Drop directly forces the network to learn representations that are invariant to the internal noise of the model, effectively smoothing the loss landscape.[53] This consistency is particularly crucial when aligning text to the static BiomedCLIP space; if the text embedding fluctuates wildly due to dropout noise, the cosine similarity matching process with the deterministic image embeddings will fail to converge.[53] By minimizing the bidirectional KL divergence, R-Drop acts as an explicit regularizer that bridges the gap between the sub-networks and the full inference model.[52]

While R-Drop requires two forward passes—theoretically doubling the forward computational time—it fundamentally bypasses the need for massive data augmentation pipelines traditionally required to stabilize text contrastive learning.[41] On the A100-40GB GPU, executing a double forward pass for a 70M parameter model introduces negligible latency. The baseline evaluation log proves that the architecture achieves a highly efficient 88,076 tokens/second at a

sequence length of 1024.[1] Therefore, the computational trade-off is highly favorable. The resulting robustness effectively eliminates the train/eval mismatch, satisfying the fourth architectural requirement and ensuring that the alignment established during training translates directly to zero-shot inference accuracy.[52]

7. Topologically Constraining the Learnable Temperature

The fifth and final architectural gap lies in the optimization of the temperature parameter (τ) within the InfoNCE contrastive loss function. In multimodal foundation models like CLIP, the temperature serves a critical topological function: it controls the sharpness of the probability distribution over the similarity scores.[67] In standard PyTorch implementations, rather than dividing by τ , it is defined mathematically as an inverse parameter, known as the logit_scale.[67]

7.1 The Mathematical Role of the Logit Scale

During contrastive learning, the core operation is computing the dot product between the image and text embeddings. Because both the query vectors from the text projection head and the key vectors from the MoCo image queue are L2-normalized, their geometric dot product represents the cosine similarity, which inherently falls within the strictly bounded range of $[-1, 1]$.[68]

When these raw dot products are fed directly into a cross-entropy loss function, a maximum logit value of 1 and a minimum of -1 provides an insufficient dynamic range. The softmax function cannot express extreme confidence or sharp categorical probability distributions with such narrow inputs.[68] To overcome this, the dot product is multiplied by the exponentiated logit_scale (where $\exp(\text{logit\_scale}) = 1/\tau$), amplifying the logits to create larger differences before the softmax operation is applied [68]:

$$\text{logits}_{i,j} = (q_i \cdot k_j) \times \exp(\text{logit\_scale})$$

7.2 The Danger of Numerical Instability and Gradient Explosion

When left as an entirely unconstrained learnable parameter, the logit_scale creates a catastrophic feedback loop. The model attempts to minimize the cross-entropy loss rapidly by pushing the logit_scale toward infinity (which is equivalent to pushing the temperature $\tau \to 0$).[72] A high scale artificially inflates the logits, generating near-one-hot probability distributions even when the actual semantic alignment between the text and image is poor.[72]

This unconstrained growth is mathematically disastrous, particularly in environments with limited batch sizes and complex medical data. As the scale grows unchecked, the gradients

passed backwards through the softmax function become highly erratic.[72] A single misaligned pair or a noisy medical label can trigger massive gradient spikes, destabilizing the projection head, propagating destructive updates into the Mamba and xLSTM layers, and causing the loss to diverge.[72] Furthermore, executing $\exp(\text{logit\_scale})$ as the parameter grows rapidly leads to NaN values due to precision overflow in mixed-precision (FP16/BF16) training environments, which are standard on modern A100 GPUs to maximize tensor core utilization.[68]

7.3 Implementing Hard Constraints and Temperature Scheduling

To guarantee numerical stability, maintain a smooth loss topology, and ensure optimal representation learning, the logit_scale must be strictly clamped. Following the structural best practices established by OpenAI's CLIP architecture, the exponentiated scale value must be mathematically bounded to a maximum of 100.[68]

In PyTorch, this is implemented as a clamped exponentiation applied after the parameter is learned but before the matrix multiplication:

$$\text{scale} = \text{clamp}(\exp(\text{logit\_scale}), \text{max} = 100)$$

By capping the scale at 100, the equivalent temperature τ is constrained to a minimum floor of 0.01 (since $1/100 = 0.01$).[68] This explicitly prevents the gradients from exploding while still providing sufficient dynamic range—expanding the dot product boundaries from $[-1, 1]$ to $[-100, 100]$—to allow the model to confidently distinguish between highly similar biomedical concepts.[68]

Additionally, the initialization of the logit_scale parameter is critical to the early stability of the training regime. Rather than initializing from zero or one, it should be initialized to match the proven starting temperature of 0.07 used extensively in MoCo and SimCLR literature.[8] Because the scale is exponentiated during the forward pass, the parameter itself should be initialized to $\ln(1/0.07) \approx 2.6592$.[69]

Python

```
# PyTorch Implementation Blueprint
self.logit_scale = nn.Parameter(torch.ones() * np.log(1 / 0.07))

# During forward pass:
scale = torch.clamp(self.logit_scale.exp(), max=100.0)
```

`logits = scale * (query_embeds @ key_embeds.T)`

Implementing this constraint instantly stabilizes the trajectory of the NT-Xent loss. When operating over the dynamic hard-negatives supplied by the MoCo v2 queue, the bounded temperature acts as a critical distance scaling factor, ensuring the model focuses on semantic alignment rather than exploiting numerical scaling shortcuts.[69]

8. Architectural Synthesis and Implementation Strategy

Addressing the five identified gaps individually is necessary, but recognizing their interdependent nature is the key to fully optimizing the Hybrid Mamba-xLSTM model. These fixes do not operate in isolation; they represent a highly cohesive mathematical system designed specifically for the rigorous demands of medical vision-language modeling.

Solving the linear projection head (Gap 2) necessitates solving the unconstrained temperature (Gap 5). A highly non-linear projection space is far more sensitive to gradient explosions; bounding the temperature ensures the MLP does not collapse due to numerical instability.[29] Simultaneously, transitioning to attention-weighted pooling (Gap 3) provides the pristine, clinical-entity-focused token representations necessary to populate the MoCo dictionary (Gap 1) with high-quality keys. If the keys are diluted by mean pooling, the massive dictionary simply becomes a repository of noise. Finally, R-Drop (Gap 4) ensures those keys remain geometrically consistent across epochs despite internal stochasticity, validating the momentum approach.[12]

8.1 Hardware-Aware VRAM Allocation

The evaluation log explicitly captures the hardware bounds: an NVIDIA A100-SXM4 with 39.5 GB usable VRAM.[1] The synthesized architectural fixes respect this ceiling perfectly.

Architectural Component	Computational Effect	VRAM Implication on A100-40GB
Hybrid Mamba-xLSTM (70M)	Base textual feature extraction	~7.67 GB peak (evaluation) [1], estimated ~22 GB during training with AdamW gradients.
MoCo v2 Dual-Queue	65,536 keys for NT-Xent loss	Negligible footprint ($\sim$33 MB for $65536 \times$ dimensions in FP32). No gradient tracking required

		for momentum key encoder.
Non-Linear MLP Projection	Dimensionality: $512 \to 512 \to$	Marginal parameter addition ($\sim$320k parameters). Negligible memory footprint.
Attention-Weighted Pooling	$V^T \tanh(Wh_i +$	Extremely lightweight feature routing ($\sim$512 parameters). Reduces $L \times$ sequence matrix to a $1 \times$ vector before heavy contrastive computation.
R-Drop Regularization	Twin forward passes per step	Increases activation memory slightly, but completely circumvents the VRAM-crushing need to double batch size for robustness.
Logit Scale Clamping	$\text{clamp}(\exp(\text{scale}), 1$	Zero memory cost. Prevents FP16 gradient overflow NaN errors.

Table 2: Hardware resource mapping of the proposed architectural fixes on the target A100-40GB GPU.

8.2 The Optimized Multimodal Convergence Paradigm

This refined architecture radically alters the model's convergence paradigm, yielding a theoretically sound pre-training loop.

In the updated flow, a biomedical text sequence (e.g., a radiology report) is processed by the interleaving Mamba and xLSTM layers, exploiting linear-time sequence modeling and selective state tracking. The resulting sequence of hidden states is aggregated via the newly implemented attention-weighted pooling, naturally emphasizing rare pathologies and clinical negations over anatomical boilerplate.[42] This highly dense 512-dimensional vector is passed through the MLP projection head, where the non-linear transformations (GELU/BatchNorm)

shed invariant textual noise, preserving the core structure within the backbone.[4]

Simultaneously, two stochastic variants of this projected embedding are generated via R-Drop's double forward pass, enforcing latent consistency via symmetric bidirectional KL divergence.[57] Finally, the stabilized, pathology-dense query embedding is contrasted against a massive, stable dictionary of historical image embeddings via the momentum-updated MoCo queue. The InfoNCE loss computation is scaled by a safely clamped temperature distribution, preventing erratic updates.[10]

By executing this integrated strategy, the 70-million parameter Hybrid Mamba-xLSTM architecture transcends the limitations of standard text encoders. It leverages mathematical stability and hardware efficiency to emerge as a highly robust, zero-shot-capable foundation model, successfully bridging the complex semantic gap between clinical language and the BiomedCLIP multimodal space without violating the strict physical limitations of the A100-40GB environment.

Works cited

1. eval_stage0_lm_1135.log
2. When are radiology reports useful for training medical image classifiers?, accessed May 3, 2026, https://multimodal-rep-learning-for-health.github.io/papers/7_When_are_radiology_reports_u.pdf
3. CLIP Model Batching When Batch Size Limited - distributed - PyTorch Forums, accessed May 3, 2026, https://discuss.pytorch.org/t/clip-model-batching-when-batch-size-limited/214427
4. A Simple Framework for Contrastive Learning of Visual Representations, accessed May 3, 2026, https://proceedings.mlr.press/v119/chen20j/chen20j.pdf
5. Exploring SimCLR: A Simple Framework for Contrastive Learning of Visual Representations, accessed May 3, 2026, https://sthalles.github.io/simple-self-supervised-learning/
6. Combining Contrastive and Reconstruction Objective for Self-Supervised Speech Representation Learning - ISCA Archive, accessed May 3, 2026, https://www.isca-archive.org/interspeech_2021/jiang21_interspeech.pdf
7. Day 4 — Popular Frameworks in Contrastive Learning | by Deepali Mishra | Medium, accessed May 3, 2026, https://medium.com/@deepsiya10/day-4-popular-frameworks-in-contrastive-learning-26543ffa3fcf
8. Review — MoCo: Momentum Contrast for Unsupervised Visual Representation Learning, accessed May 3, 2026, https://sh-tsang.medium.com/review-moco-momentum-contrast-for-unsupervised-visual-representation-learning-99b590c042a9
9. Contrastive Learning for Unsupervised Auditory Texture Models - ScholarWorks@UARK, accessed May 3, 2026,

https://scholarworks.uark.edu/cgi/viewcontent.cgi?article=1096&context=csceuht
10. Momentum Contrast for Unsupervised Visual Representation Learning - CVF Open Access, accessed May 3, 2026, https://openaccess.thecvf.com/content_CVPR_2020/papers/He_Momentum_Contrast_for_Unsupervised_Visual_Representation_Learning_CVPR_2020_paper.pdf
11. Intriguing Properties of Contrastive Losses, accessed May 3, 2026, https://neurips.cc/media/neurips-2021/Slides/28477.pdf
12. 39: MoCO v1 & v2 Explained - Casual GAN Papers, accessed May 3, 2026, https://www.casualganpapers.com/self-supervised-representation-learning-encoder/MoCo-explained.html
13. Making the Most of Text Semantics to Improve Biomedical Vision–Language Processing, accessed May 3, 2026, https://www.ecva.net/papers/eccv_2022/papers_ECCV/papers/136960001.pdf
14. Mask & Match: Learning to Recognize Handwritten Math with Self-Supervised Attention, accessed May 3, 2026, https://arxiv.org/html/2508.06107v1
15. Momentum contrast for Unsupervised representation learning (MoCo) - CMP, accessed May 3, 2026, https://cmp.felk.cvut.cz/~toliageo/rg/papers/slides/moco.pdf
16. MoCo v2 — MMSelfSup 1.0.0 documentation, accessed May 3, 2026, https://mmselfsup.readthedocs.io/en/latest/papers/mocov2.html
17. Momentum Contrastive Learning with Enhanced Negative Sampling and Hard Negative Filtering - arXiv, accessed May 3, 2026, https://arxiv.org/html/2501.16360v1
18. Easily Explained: Momentum Contrast for Unsupervised Visual Representation Learning | by Hey Amit | Medium, accessed May 3, 2026, https://medium.com/@heyamit10/easily-explained-momentum-contrast-for-unsupervised-visual-representation-learning-10bf51e08fb1
19. Review — MoCo v2: Improved Baselines with Momentum Contrastive Learning, accessed May 3, 2026, https://sh-tsang.medium.com/review-moco-v2-improved-baselines-with-momentum-contrastive-learning-8b03598a0324
20. Align before Fuse: Vision and Language Representation Learning with Momentum Distillation - NIPS papers, accessed May 3, 2026, https://proceedings.neurips.cc/paper/2021/file/505259756244493872b7709a8a01b536-Paper.pdf
21. Momentum Contrastive Learning - YouTube, accessed May 3, 2026, https://www.youtube.com/watch?v=LvHwBQF14zs
22. Easily Explained: Momentum Contrast for Unsupervised Visual Representation Learning (MoCo) | by Nour Eldin Alaa | Medium, accessed May 3, 2026, https://medium.com/@nour_badr/easily-explained-momentum-contrast-for-unsupervised-visual-representation-learning-moco-c6f00a95c4b2
23. MoCo-v2 in PyTorch | SimCLR with Computational Constraints - Analytics Vidhya, accessed May 3, 2026, https://www.analyticsvidhya.com/blog/2020/08/moco-v2-in-pytorch/
24. arXiv:2002.05709v3 [cs.LG] 1 Jul 2020, accessed May 3, 2026,

https://arxiv.org/pdf/2002.05709
25. Papers Explained 200: SimCLR - Ritvik Rastogi - Medium, accessed May 3, 2026, https://ritvik19.medium.com/papers-explained-200-simclr-191ecf19d2fc
26. Self-supervised learning tutorial: Implementing SimCLR with pytorch lightning | AI Summer, accessed May 3, 2026, https://theaisummer.com/simclr/
27. Projection Head is Secretly an Information Bottleneck - OpenReview, accessed May 3, 2026, https://openreview.net/forum?id=L0evcuybH5
28. The Mechanism of Prediction Head in Non-contrastive Self-supervised Learning, accessed May 3, 2026, https://proceedings.neurips.cc/paper_files/paper/2022/file/9d276b0a087efdd2404f3295b26c24c1-Paper-Conference.pdf
29. Exploring Self-Supervised Learning Through SimCLR: Reproducing and Evaluating Natural Image Classification and Transfer Learning - Uni Bamberg, accessed May 3, 2026, https://www.uni-bamberg.de/fileadmin/xai/studies/theses/2025/Bachelor_Thesis_Sascha_Wolf.pdf
30. Understanding and Improving the Role of Projection Head in Self-Supervised Learning - arXiv, accessed May 3, 2026, https://arxiv.org/pdf/2212.11491
31. A Contrastive Representation Learning Method for Event Classification in Φ-OTDR Systems, accessed May 3, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12349413/
32. lightly/lightly/models/modules/heads.py at master · lightly-ai/lightly - GitHub, accessed May 3, 2026, https://github.com/lightly-ai/lightly/blob/master/lightly/models/modules/heads.py
33. Investigating the Benefits of Projection Head for Representation Learning - OpenReview, accessed May 3, 2026, https://openreview.net/forum?id=GgEAdqYPNA
34. CultureCLIP: Empowering CLIP with Cultural Awareness through Synthetic Images and Contextualized Captions - arXiv, accessed May 3, 2026, https://arxiv.org/html/2507.06210v1
35. Brief Review — Align before Fuse: Vision and Language Representation Learning with Momentum Distillation - Sik-Ho Tsang, accessed May 3, 2026, https://sh-tsang.medium.com/brief-review-align-before-fuse-vision-and-language-representation-learning-with-momentum-34386305ce0c
36. Word embeddings as autonomous predictors in materials design—the effect of inherent variability on information transfer - PMC, accessed May 3, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC12888715/
37. Learning to Exploit Temporal Structure for Biomedical Vision-Language Processing - Microsoft Research, accessed May 3, 2026, https://www.microsoft.com/en-us/research/publication/learning-to-exploit-temporal-structure-for-biomedical-vision-language-processing/
38. Comprehensive language–image pre-training for 3D medical image understanding - arXiv, accessed May 3, 2026, https://arxiv.org/html/2510.15042v2
39. Vision-Language Modelling for Radiological Imaging and Reports in the Low Data Regime, accessed May 3, 2026,

https://proceedings.mlr.press/v227/windsor24a/windsor24a.pdf
40. Medical Vision Language Pretraining: A survey - arXiv, accessed May 3, 2026, https://arxiv.org/html/2312.06224v1
41. Contrastive Representation Learning | Lil'Log, accessed May 3, 2026, https://lilianweng.github.io/posts/2021-05-31-contrastive/
42. BioViL-T - Temporal Vision-Language Model for Radiology - AIKosh, accessed May 3, 2026, https://aikosh.indiaai.gov.in/home/models/details/biovil_t_temporal_vision_language_model_for_radiology.html
43. A Review of Longitudinal Radiology Report Generation: Dataset Composition, Methods, and Performance Evaluation - arXiv, accessed May 3, 2026, https://arxiv.org/html/2510.12444v1
44. [2301.04558] Learning to Exploit Temporal Structure for Biomedical Vision-Language Processing - arXiv, accessed May 3, 2026, https://arxiv.org/abs/2301.04558
45. Combining Graph Attention Mechanism and PageRank to Learn Graph-level Representations - SFU Summit, accessed May 3, 2026, https://summit.sfu.ca/_flysystem/fedora/2023-01/etd21979.pdf
46. Underwater Diffusion Attention Network with Contrastive Language-Image Joint Learning for Underwater Image Enhancement - arXiv, accessed May 3, 2026, https://arxiv.org/html/2505.19895v1
47. Chest X-Ray Report Generation Using Abnormality Guided Vision Language Model - IEEE Xplore, accessed May 3, 2026, https://ieeexplore.ieee.org/iel8/6287639/10820123/11153468.pdf
48. Organ-Aware Attention Improves CT Triage and Classification - arXiv, accessed May 3, 2026, https://arxiv.org/html/2601.13385v1
49. DMAPLM: A multimodal pretrained framework for computational drug repositioning - Research journals - PLOS, accessed May 3, 2026, https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1014192&rev=1
50. Ranking attention multiple instance learning for lymph node metastasis prediction on multicenter cervical cancer MRI - PMC, accessed May 3, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11633800/
51. Learning Generalized Medical Image Representations Through Image-Graph Contrastive Pretraining - Proceedings of Machine Learning Research, accessed May 3, 2026, https://proceedings.mlr.press/v225/khanna23a/khanna23a.pdf
52. R-Drop: Regularized Dropout for Neural Networks, accessed May 3, 2026, https://papers.neurips.cc/paper_files/paper/2021/file/5a66b9200f29ac3fa0ae244cc2a51b39-Paper.pdf
53. Research on Entity Pair Relation Classification Based on Contrastive Learning and Biaffine Model - Preprints.org, accessed May 3, 2026, https://www.preprints.org/manuscript/202503.1462
54. [2106.14448] R-Drop: Regularized Dropout for Neural Networks - arXiv, accessed May 3, 2026, https://arxiv.org/abs/2106.14448
55. Entity Pair Relation Classification Based on Contrastive Learning and Biaffine

Model - IEEE Xplore, accessed May 3, 2026, https://ieeexplore.ieee.org/iel8/6287639/10820123/11095676.pdf
56. Entity Pair Relation Classification Based on Contrastive Learning and Biaffine Model - IEEE Xplore, accessed May 3, 2026, https://ieeexplore.ieee.org/iel8/6287639/6514899/11095676.pdf
57. R-Drop: Regularized Dropout for Neural Networks - arXiv, accessed May 3, 2026, https://arxiv.org/pdf/2106.14448
58. R-Drop: A simple and effective regular method to correct the defects of Dropout - Microsoft, accessed May 3, 2026, https://www.microsoft.com/en-us/research/articles/r-drop-a-simple-and-effective-regular-method-to-correct-the-defects-of-dropout/
59. Medical prediction from missing data with max-minus negative regularized dropout - PMC, accessed May 3, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC10373302/
60. KL-Divergence Explained: Intuition, Formula, and Examples - DataCamp, accessed May 3, 2026, https://www.datacamp.com/tutorial/kl-divergence
61. Finally! A Clear Derivation of the VAE KL Loss | by Jae H. Park | Medium, accessed May 3, 2026, https://medium.com/@jpark7/finally-a-clear-derivation-of-the-vae-kl-loss-4cb38d2e47b3
62. Kullback–Leibler divergence - Wikipedia, accessed May 3, 2026, https://en.wikipedia.org/wiki/Kullback%E2%80%93Leibler_divergence
63. How to Calculate KL Divergence in R - GeeksforGeeks, accessed May 3, 2026, https://www.geeksforgeeks.org/r-language/how-to-calculate-kl-divergence-in-r/
64. Team Bohan Li at PAN: DeBERTa-v3 with R-Drop Regularization for Human-AI Collaborative Text Classification, accessed May 3, 2026, https://ceur-ws.org/Vol-4038/paper_299.pdf
65. Deciphering the Projection Head: Representation Evaluation Self-supervised Learning - IJCAI, accessed May 3, 2026, https://www.ijcai.org/proceedings/2024/0522.pdf
66. LoRA Meets Dropout under a Unified Framework - arXiv, accessed May 3, 2026, https://arxiv.org/html/2403.00812v2
67. Building a Tiny CLIP: Contrastive Image-Text Learning from Scratch - Medium, accessed May 3, 2026, https://medium.com/@varun_54675/building-a-tiny-clip-contrastive-image-text-learning-from-scratch-0ca31bf40c65
68. Two questions regarding model implementation: · Issue #48 · openai/CLIP - GitHub, accessed May 3, 2026, https://github.com/openai/CLIP/issues/48
69. Difference between `logit_scale` initialisation in Transformers CLIP and the original OpenAI implementation. · Issue #13430 - GitHub, accessed May 3, 2026, https://github.com/huggingface/transformers/issues/13430
70. Temperature Clipping Missing · Issue #46 · openai/CLIP - GitHub, accessed May 3, 2026, https://github.com/openai/CLIP/issues/46
71. arXiv:2207.01297v4 [cs.CV] 26 Mar 2023, accessed May 3, 2026, https://arxiv.org/pdf/2207.01297

72. Analyzing the Impact of Learnable Softmax Temperature in Contrastive Visual-Textual Alignment Systems: Benefits, Drawbacks, and Alternative Approaches | OpenReview, accessed May 3, 2026, https://openreview.net/forum?id=rx1QNhsNsK
73. MM-TS: Multi-Modal Temperature and Margin Schedules for Contrastive Learning with Long-Tail Data - CVF Open Access, accessed May 3, 2026, https://openaccess.thecvf.com/content/WACV2026/papers/Sheludzko_MM-TS_Multi-Modal_Temperature_and_Margin_Schedules_for_Contrastive_Learning_with_WACV_2026_paper.pdf
74. ssnl/moco_align_uniform: MoCo with Alignment and Uniformity Loss. - GitHub, accessed May 3, 2026, https://github.com/ssnl/moco_align_uniform